

Mục lục

1	Bài 1: Dãy Dao	3
1.1	Đề bài	3
1.2	Lời giải	4
1.2.1	Nhận xét quan trọng	4
1.2.2	Subtask 1 (20% số điểm, $N, T, Q \leq 1000$).	4
1.2.3	Subtask 2 (20% số điểm, $s_{i,j}, A \leq 10$).	4
1.2.4	Subtask 3 (20% số điểm, $N \leq 2$).	4
1.2.5	Subtask 4 (20% số điểm, $N \leq 25$).	4
1.2.6	Subtask 5 (20% số điểm, không có ràng buộc thêm) – Lời giải đầy đủ.	5
2	Bài 2: Chọn Đội	8
2.1	Đề bài	8
2.2	Lời giải	9
2.2.1	Nhận xét quan trọng	9
2.2.2	Subtask 1 (25% số điểm, $N \leq 12$)	9
2.2.3	Subtask 2 (25% số điểm, $N \leq 30, T \leq 1000$)	9
2.2.4	Subtask 3 (25% số điểm, $N \leq 200, T \leq 5000$)	10
2.2.5	Subtask 4 (25% số điểm, không có ràng buộc thêm) – Lời giải đầy đủ	10
3	Bài 3: Tuần Tra	15
3.1	Đề bài	15
3.2	Lời giải	15
3.2.1	Nhận xét quan trọng	15
3.2.2	Subtask 1 (20% số điểm, $N, Q \leq 200$)	16
3.2.3	Subtask 2 (20% số điểm, $N \leq 1000$)	16
3.2.4	Subtask 3 (20% số điểm, mọi truy vấn đều có khoảng cách xuất phát luôn đủ xa để đạt độ an toàn lớn nhất của cây)	17
3.2.5	Subtask 4 (20% số điểm, $Q \leq 200$)	17
3.2.6	Subtask 5 (20% số điểm, không ràng buộc thêm) – Lời giải đầy đủ	18

Thông tin

Tác giả

- **Hoàng Hồ Minh Quang** - THPT Chuyên Bình Long, ngành Khoa Học Máy Tính trường Đại học Khoa Học Tự Nhiên Đại Học Quốc Gia Hồ Chí Minh

Đơn vị tài trợ

Trung tâm tin học. **Code Prep Pros** - là trung tâm luyện thi lập trình dành cho học sinh từ cấp tỉnh đến quốc gia, được thiết kế bởi các VOIer giàu kinh nghiệm. Chương trình cung cấp kiến thức vững chắc, kỹ năng giải quyết vấn đề và chiến lược làm bài tối ưu, giúp học sinh tự tin chinh phục các kỳ thi lập trình.

Group nộp bài, trao đổi và sửa đề thi Tin Học Trẻ, HSG Tỉnh, Chọn DTQG, vào chuyên tin 10

Tham gia ngay cộng đồng chia sẻ tài liệu, đề thi và lời giải cho kỳ thi vào lớp 10 chuyên Tin, Tin Học Trẻ bảng B - C, học sinh giỏi Tỉnh và học sinh giỏi Quốc Gia. Cơ hội rèn luyện, học hỏi từ tất cả các bạn trên Cả nước. [Bấm vào đây!](#)



§1 Bài 1: Đếm Đảo

§1.1 Đề bài

Cho N vùng biển đánh số $1 \dots N$, vùng i có c_i hòn đảo với diện tích $s_{i,1}, s_{i,2}, \dots, s_{i,c_i}$. Có Q truy vấn, mỗi truy vấn cho ngưỡng A . Với mỗi A , ta chỉ xét những đảo có diện tích **ít nhất** A . Nhiệm vụ là tìm vùng biển có **nhiều đảo được xét nhất**; nếu có nhiều vùng cùng số lượng thì chọn vùng có **chỉ số lớn nhất**.

Input

- Dữ liệu vào: file ISLANDS.INP;
- Dòng đầu: Ghi N, Q ($1 \leq N, Q \leq 10^5$);
- N dòng tiếp: Mỗi dòng bắt đầu bởi c_i ($1 \leq c_i \leq 3 \cdot 10^5$) rồi đến c_i số $s_{i,j}$ ($1 \leq s_{i,j} \leq 10^9$);
- Q dòng cuối: Mỗi dòng ghi một số A ($1 \leq A \leq 10^9$);
- Tổng số đảo $T = \sum c_i \leq 3 \cdot 10^5$.

Output

- Kết quả: Ghi ra file ISLANDS.OUT gồm Q dòng, mỗi dòng là chỉ số vùng biển thỏa mãn tương ứng với mỗi truy vấn.

§1.2 Lời giải

§1.2.1 Nhận xét quan trọng

Với vùng i và ngưỡng A , gọi $cnt_i(A)$ là số đảo của vùng i có diện tích $\geq A$. Bài toán yêu cầu tìm i sao cho cặp $(cnt_i(A), i)$ là lớn nhất theo thứ tự từ điển (so sánh số đảo trước, nếu bằng thì so chỉ số). Nhận xét này cho phép ta mô hình hóa việc so sánh theo cặp, ví dụ như bằng `pair<int, int>` trong C++.

Vì tổng số đảo $T \leq 3 \cdot 10^5$, các thao tác trên toàn bộ đảo như sắp xếp, duyệt một lượt là hoàn toàn khả thi. Thách thức chính là Q có thể lên đến 10^5 , đòi hỏi mỗi truy vấn phải được trả lời nhanh (cận $O(\log T)$ hoặc $O(1)$ sau tiền xử lý).

§1.2.2 Subtask 1 (20% số điểm, $N, T, Q \leq 1000$).

Tiếp cận. Với mỗi truy vấn A , ta khởi tạo mảng đếm $cnt[1 \dots N] = 0$. Duyệt toàn bộ T hòn đảo: nếu đảo thuộc vùng i có diện tích $s \geq A$ thì tăng $cnt[i]$ lên 1. Sau khi duyệt xong, duyệt $i = 1 \dots N$, chọn i có cặp $(cnt[i], i)$ lớn nhất (so sánh bằng `pair` hoặc `if` thủ công).

Độ phức tạp. Mỗi truy vấn duyệt $O(T)$ đảo và $O(N)$ vùng, tổng thời gian $O(Q(T + N))$. Bộ nhớ $O(N)$.

§1.2.3 Subtask 2 (20% số điểm, $s_{i,j}, A \leq 10$).

Tiếp cận. Do mọi diện tích và ngưỡng đều ≤ 10 , ta có thể **tiền xử lý** đáp án cho tất cả 10 giá trị A có thể gặp. Với mỗi $a = 1 \dots 10$, ta duyệt toàn bộ đảo, nếu $s \geq a$ thì tăng biến đếm cho vùng tương ứng. Sau khi duyệt xong, chọn vùng thắng cho ngưỡng a và lưu vào mảng `winner[a]`. Mỗi truy vấn chỉ cần in `winner[A]`.

Độ phức tạp. Tiền xử lý: $O(10 \cdot T + 10 \cdot N) = O(T + N)$. Mỗi truy vấn $O(1)$. Tổng thời gian $O(T + N + Q)$, bộ nhớ $O(1)$ cho mảng `winner`.

§1.2.4 Subtask 3 (20% số điểm, $N \leq 2$).

Tiếp cận. Với số vùng rất ít, ta có thể trả lời mỗi truy vấn bằng tìm kiếm nhị phân. Sắp xếp riêng danh sách diện tích của từng vùng theo thứ tự tăng dần. Với truy vấn A , dùng `lower_bound` lên vùng i để tìm vị trí đầu tiên có diện tích $\geq A$. Khi đó $cnt_i(A) =$ số đảo vùng i – vị trí đó. Tính $cnt_i(A)$ cho cả N vùng rồi chọn vùng có cặp $(cnt_i(A), i)$ lớn hơn.

Chứng minh tính đúng đắn. Sắp xếp không làm thay đổi tập đảo. `lower_bound` trả về chỉ số phần tử nhỏ nhất $\geq A$, số phần tử $\geq A$ chính là số phần tử từ vị trí đó đến hết mảng.

Độ phức tạp. Sắp xếp: $O(T \log T)$. Mỗi truy vấn: $O(\log T)$ cho mỗi vùng, với $N \leq 2$ là $O(\log T)$. Tổng thời gian $O(T \log T + Q \log T)$. Bộ nhớ $O(T)$.

§1.2.5 Subtask 4 (20% số điểm, $N \leq 25$).

Tiếp cận. Mở rộng ý tưởng của subtask 3 cho $N \leq 25$: vẫn sắp xếp danh sách đảo của từng vùng. Với mỗi truy vấn, lặp qua tất cả N vùng, dùng `lower_bound` để tính $cnt_i(A)$, đồng thời duy trì `best` là cặp (cnt, id) lớn nhất đã thấy.

Chứng minh tính đúng đắn. Tương tự subtask 3, áp dụng cho N vùng.

Độ phức tạp. Sắp xếp $O(T \log T)$. Mỗi truy vấn $O(N \log T)$. Tổng thời gian $O(T \log T + Q \cdot N \log T)$. Với $N \leq 25$, $Q \leq 10^5$, $T \leq 3 \cdot 10^5$, khoảng $25 \cdot 10^5 \cdot \log(3 \cdot 10^5) \approx 4.5 \cdot 10^7$. Bộ nhớ $O(T)$.

§1.2.6 Subtask 5 (20% số điểm, không có ràng buộc thêm) – Lời giải đầy đủ.**Ý tưởng / Tiếp cận.**

Ở subtask 4, mỗi truy vấn ta phải duyệt lại tất cả N vùng, gây ra chi phí $O(N \log T)$ cho mỗi truy vấn – không đủ tốt khi N, Q lên tới 10^5 . Ta nhận thấy: khi ngưỡng A **giảm dần**, các đảo đủ điều kiện sẽ được **thêm vào** chứ không bị loại bỏ. Điều này gợi ý xử lý **offline**:

- Sắp xếp tất cả đảo theo diện tích **giảm dần**.
- Sắp xếp các truy vấn theo A **giảm dần**.
- Dùng kỹ thuật hai con trỏ (two-pointer): duy trì con trỏ **ptr** trên mảng đảo đã sắp, và mảng **cnt[i]** là số đảo hiện tại của vùng i đã được thêm. Với mỗi truy vấn A , ta thêm tất cả đảo có diện tích $\geq A$ (từ vị trí **ptr** trở đi) vào vùng tương ứng, cập nhật **cnt**. Sau khi thêm đủ, đáp án cho truy vấn này chính là chỉ số của vùng có cặp $(cnt[i], i)$ lớn nhất.

Câu hỏi: Làm sao để biết nhanh vùng nào đang có nhiều đảo nhất? Đơn giản như sau: khi một vùng i được thêm đảo mới, **cnt[i]** tăng lên 1, do đó cặp $(cnt[i], i)$ có thể vượt qua “kỷ lục” hiện tại hoặc không. Vì vậy, ta chỉ cần duy trì biến **best = (bestCnt, bestId)** là cặp lớn nhất **tại thời điểm hiện tại**. Mỗi khi tăng **cnt[i]**, ta so sánh và cập nhật **best = max(best, {cnt[i], i})**.

Nhờ thế, sau khi thêm hết đảo cần thiết cho truy vấn, **best.second** chính là đáp án.

Các bước của thuật toán.

1. Đọc dữ liệu, lưu các đảo dưới dạng cấu trúc **{area, region_id}**.
2. Lưu các truy vấn kèm chỉ số gốc.
3. Sắp xếp mảng đảo giảm dần theo **area**.
4. Sắp xếp mảng truy vấn giảm dần theo A .
5. Khởi tạo mảng **cnt** kích thước $N + 1$ bằng 0, **best = {0, n}**.
6. Duyệt từng truy vấn:
 - Trong khi **ptr < T** và **islands[ptr].area >= A**:
`region = islands[ptr].id;`
`cnt[region]++;`
`best = max(best, {cnt[region], region});`
`ptr++.`
 - Gán **ans[query.original_idx] = best.second**.
7. In mảng **ans** theo thứ tự ban đầu.

Chứng minh tính đúng đắn.

- Tính đúng của việc thêm đảo: vì mảng đảo sắp giảm dần, khi gặp truy vấn A nhỏ hơn hoặc bằng các truy vấn trước đó (do sắp giảm dần), tất cả đảo có **area >= A** đều đã được thêm hoặc sẽ được thêm nốt trong bước **while**. Điều kiện **islands[ptr].area >= A** đảm bảo chỉ thêm những đảo thỏa mãn.

- Tính đúng của biến **best**: Ban đầu chưa có đảo nào, $\text{cnt}[i] = 0$ với mọi i , do đó vùng có chỉ số lớn nhất là vùng n sẽ thắng. Mỗi khi một vùng tăng cnt , ta so sánh với **best** hiện tại. Vì tất cả cnt khác không đổi, nếu vùng vừa tăng vượt qua **best** thì **best** được cập nhật; ngược lại **best** vẫn là giá trị lớn nhất trước đó. Sau khi đã thêm mọi đảo thỏa mãn, **best** chính là cặp tối ưu trong toàn bộ N vùng.
- Thứ tự truy vấn giảm dần đảm bảo ta chỉ thêm đảo, không bao giờ bớt, phù hợp với việc duy trì **best** một cách đơn giản.

Độ phức tạp.

- Sắp xếp đảo: $O(T \log T)$.
- Sắp xếp truy vấn: $O(Q \log Q)$.
- Duyệt hai con trỏ: mỗi đảo được thêm đúng một lần, mỗi truy vấn xét $O(1)$, tổng $O(T + Q)$.
- Thời gian tổng: $O(T \log T + Q \log Q)$, dễ dàng qua được giới hạn $T \leq 3 \cdot 10^5, Q \leq 10^5$.
- Bộ nhớ: $O(N + T + Q)$, rất nhỏ so với giới hạn.

¶ Mã nguồn minh họa (C++).

```

1 #include <bits/stdc++.h> // NeOWami
2 using namespace std;
3
4 const int N = 1e5 + 5;
5 int n, q;
6 int cnt[N], ans[N];
7 struct Island {
8     int area, id;
9     bool operator<(const Island& o) const {
10         return area > o.area;
11     }
12 };
13
14 struct Query {
15     int a, idx;
16     bool operator<(const Query& o) const {
17         return a > o.a;
18     }
19 };
20
21 signed main() {
22     cin.tie(NULL) -> sync_with_stdio(false); cout.tie(NULL);
23     if (ifstream("ISLANDS.INP")) {
24         freopen("ISLANDS.INP", "r", stdin);
25         freopen("ISLANDS.OUT", "w", stdout);
26     }
27
28     int n, q; cin >> n >> q;
29     vector<Island> islands;
30     for (int i = 1; i <= n; ++i) {
31         int c; cin >> c;

```

```
32     while (c--) {
33         int s; cin >> s;
34         islands.push_back({s, i});
35     }
36 }
37
38 vector<Query> queries(q);
39 for (int i = 0; i < q; ++i) {
40     cin >> queries[i].a;
41     queries[i].idx = i;
42 }
43
44 sort(islands.begin(), islands.end());
45 sort(queries.begin(), queries.end());
46
47 pair<int,int> best = {0, n};
48 int ptr = 0;
49
50 for (Query &qr : queries) {
51     while (ptr < islands.size() && islands[ptr].area >= qr.a) {
52         int reg = islands[ptr].id;
53         cnt[reg]++;
54         best = max(best, make_pair(cnt[reg], reg));
55         ++ptr;
56     }
57     ans[qr.idx] = best.second;
58 }
59
60 for (int i = 0; i < q; i++) cout << ans[i] << '\n';
61 return 0;
62 }
```

CODE PREP PROS
EXPERIENCE-EFFORT-ACHIEVEMENT

§2 Bài 2: Chọn Đội

§2.1 Đề bài

Cho N học sinh, mỗi học sinh i có kỹ năng S_i và độ nổi tiếng P_i . Cần chọn ra hai đội (mỗi đội có ít nhất một học sinh, không giao nhau, không bắt buộc chọn hết) sao cho tổng kỹ năng hai đội bằng nhau. Đội trưởng của một đội là học sinh có độ nổi tiếng lớn nhất trong đội đó. Độ hấp dẫn trận đấu là $|P_{\text{đội trưởng 1}} - P_{\text{đội trưởng 2}}|$. Yêu cầu tìm độ hấp dẫn lớn nhất có thể.

Input (Tập văn bản TEAMS.INP)

- Dòng 1: Ghi số nguyên dương N ($1 \leq N \leq 1000$);
- Dòng 2: Ghi N số nguyên $S_1, S_2, \dots, S_N$ ($1 \leq S_i$), với tổng kỹ năng $T = \sum S_i \leq 10^5$;
- Dòng 3: Ghi N số nguyên $P_1, P_2, \dots, P_N$ ($1 \leq P_i \leq 10^9$).

Output (Tập văn bản TEAMS.OUT)

- Ghi một số nguyên duy nhất là độ hấp dẫn lớn nhất (dữ liệu luôn đảm bảo tồn tại cách chọn).

Sample Input 1

```
8
4 7 3 8 5 6 9 2
15 3 20 11 8 30 6 25
```

Sample Output 1

```
24
```

CODE PREP PROS
EXPERIENCE-EFFORT-ACHIEVEMENT

§2.2 Lời giải

§2.2.1 Nhận xét quan trọng

Để quản lí đội trưởng một cách thuận tiện, ta **sắp xếp học sinh theo P_i không giảm** và đánh lại chỉ số từ 1 đến N theo thứ tự đó. Khi ấy, với một tập học sinh bất kì, đội trưởng (có P lớn nhất) có thể được coi là **học sinh có chỉ số lớn nhất** trong tập đó. Nếu có nhiều học sinh cùng P , ta vẫn có thể chọn người ngoài cùng bên phải làm đại diện mà không ảnh hưởng đến giá trị P của đội trưởng.

Giả sử hai đội đã được chọn, gọi low và $high$ lần lượt là chỉ số của hai đội trưởng, với $low < high$ (nếu $P_{low} = P_{high}$, ta vẫn gán chỉ số nhỏ hơn cho low). Hiển nhiên độ hấp dẫn bằng $P_{high} - P_{low}$. Từ tính chất sắp xếp, ta suy ra ba ràng buộc quan trọng:

- Mọi học sinh có chỉ số $k > high$ **không được chọn** vào bất kì đội nào (vì nếu được chọn, đội trưởng của đội đó sẽ có chỉ số lớn hơn $high$).
- Những học sinh có chỉ số $low < k < high$ **chỉ có thể** vào đội có đội trưởng là $high$, hoặc bị bỏ qua (vì nếu vào đội của low , chỉ số lớn nhất của đội đó sẽ vượt quá low).
- Những học sinh có chỉ số $k < low$ có thể vào một trong hai đội hoặc không được chọn.

Với mỗi cặp $(low, high)$ thỏa mãn, ta chỉ cần kiểm tra xem có thể chọn một tập con các học sinh (tuân thủ ràng buộc trên) sao cho tổng kỹ năng hai đội bằng nhau hay không. Bài toán trở thành: tìm cặp $(low, high)$ có $P_{high} - P_{low}$ lớn nhất mà tồn tại cách chia thỏa mãn.

Đặt $T = \sum_{i=1}^N S_i \leq 10^5$. Đây là con số khá nhỏ, cho phép ta sử dụng các thuật toán quy hoạch động với tổng làm tham số.

§2.2.2 Subtask 1 (25% số điểm, $N \leq 12$)

¶ **Tiếp cận.** Duyệt toàn bộ 3^N cách gán mỗi học sinh vào đội 1, đội 2, hoặc không chọn. Với mỗi cách gán, tính tổng kỹ năng sum_1, sum_2 và xác định đội trưởng $(\max P_1, \max P_2)$. Chỉ xét những cách mà cả hai đội không rỗng và $sum_1 = sum_2$, rồi cập nhật đáp án với $|\max P_1 - \max P_2|$.

¶ **Độ phức tạp.** Có 3^N mask, mỗi mask cần $O(N)$ để tính tổng và tìm $\max P$. Thời gian $O(N \cdot 3^N)$, với $N = 12$ vào khoảng $12 \cdot 531441 \approx 6.3 \cdot 10^6$ phép toán. Bộ nhớ $O(N)$.

§2.2.3 Subtask 2 (25% số điểm, $N \leq 30, T \leq 1000$)

¶ **Tiếp cận.** Duyệt tất cả các cặp $(low, high)$ với $1 \leq low < high \leq N$. Với mỗi cặp, sử dụng quy hoạch động hiệu (difference DP) để kiểm tra khả năng cân bằng. Gọi $d = sum_1 - sum_2$ là hiệu tổng kỹ năng hai đội. Khởi tạo $d = S_{low} - S_{high}$ (hai đội trưởng đương nhiên thuộc đội tương ứng). Dùng mảng **reachable** kích thước $2T + 1$ với offset T (nhằm để lưu khi d âm) để biểu diễn các giá trị d có thể đạt được.

Duyệt lần lượt các học sinh khác:

- Với $k < low$: có 3 lựa chọn – bỏ qua (giữ d), cho vào đội 1 ($d \rightarrow d + S_k$), cho vào đội 2 ($d \rightarrow d - S_k$).
- Với $low < k < high$: chỉ có 2 lựa chọn – bỏ qua, hoặc cho vào đội 2 ($d \rightarrow d - S_k$).

Sau khi duyệt hết, nếu `reachable[T]` (tương ứng $d = 0$) là đúng thì cặp $(low, high)$ khả thi.

¶ **Độ phức tạp.** Có $O(N^2)$ cặp, mỗi cặp duyệt $O(N)$ học sinh, với mỗi học sinh cập nhật mảng $O(T)$. Tổng thời gian $O(N^3 \cdot T)$. Với $N = 30, T = 1000$, số phép tính cỡ $30^3 \cdot 1000 = 27 \cdot 10^6$, đủ để vượt qua subtask. Bộ nhớ $O(T)$.

§2.2.4 Subtask 3 (25% số điểm, $N \leq 200, T \leq 5000$)

¶ **Tiếp cận.** Ta tiền xử lý một mảng DP cho phần trước low : $pref[i][d]$ ($0 \leq d \leq T$) là `true` nếu có thể chọn một số học sinh trong $1..i$ và chia thành hai đội sao cho trị tuyệt đối của hiệu tổng kỹ năng giữa hai đội là d . Công thức chuyển trạng thái: khi thêm S_i , từ d có thể giữ nguyên d , hoặc tăng d lên $d + S_i$, hoặc giảm còn $|d - S_i|$. Khi cố định low , ta duy trì một mảng $mid[s]$ là `true` nếu có thể chọn một tập con từ các học sinh nằm giữa low và $high$ (không bao gồm $high$) có tổng kỹ năng bằng s . Mỗi khi tăng $high$ lên 1, ta cập nhật mid bằng cách thêm S_{high-1} vào tập hiện tại (knapsack). Với mỗi cặp $(low, high)$, tồn tại cách chọn hợp lệ nếu có s sao cho $mid[s] = \text{true}$ và $pref[low-1][s + S_{high} - S_{low}] = \text{true}$. Giải thích: Gọi A là tổng kỹ năng đội 1 lấy từ phần trước low , B là tổng đội 2 lấy từ phần trước low . Ta có $|A - B| = d$. Tổng kỹ năng đội 1 $= A + S_{low}$, tổng đội 2 $= B + s + S_{high}$ (với s là tổng của tập giữa). Để hai đội bằng nhau: $A - B = s + S_{high} - S_{low}$. Do đó cần $d = |s + S_{high} - S_{low}|$.

¶ Các bước của thuật toán.

- Sắp xếp học sinh theo P tăng dần.
- Tính mảng $pref$: $pref[0][0] = \text{true}$, sau đó duyệt $i = 1..N$, cập nhật $pref[i]$ từ $pref[i-1]$.
- Duyệt low từ 1 đến N :
 - Khởi tạo mid toàn `false`, $mid[0] = \text{true}$.
 - Duyệt $high$ từ $low + 1$ đến N :
 - Với mỗi s có $mid[s] = \text{true}$, kiểm tra nếu $pref[low-1][|s + S_{high} - S_{low}|]$ đúng thì cập nhật đáp án với $P_{high} - P_{low}$.
 - Sau đó thêm S_{high} vào mid (vì $high$ sẽ trở thành “ở giữa” cho các $high$ tiếp theo), tức cập nhật mid theo kiểu knapsack: thêm S_{high} vào các tổng hiện có.
- In ra đáp án lớn nhất tìm được.

¶ **Độ phức tạp.** Tiền xử lý $pref$ mất $O(NT)$. Vòng lặp ngoài low có N lần, bên trong duyệt $high$ $O(N)$ lần, mỗi bước duyệt s từ 0 đến T và cập nhật mid $O(T)$. Tổng thời gian $O(N^2T)$. Với $N = 200, T = 5000$, $200^2 \cdot 5000 = 2 \cdot 10^8$. Bộ nhớ $O(NT)$ cho $pref$, khoảng $200 \cdot 5001 \approx 10^6$ phần tử.

§2.2.5 Subtask 4 (25% số điểm, không có ràng buộc thêm) – Lời giải đầy đủ

¶ **Ý tưởng / Tiếp cận.** Ta có thể cải tiến từ subtask 3 bằng cách tránh duyệt tuần tự $high$. Nhận xét: khi cố định low , ta chỉ quan tâm đến $high$ có chỉ số lớn nhất mà vẫn tồn tại cách chọn hợp lệ, vì P_{high} càng lớn thì độ hấp dẫn càng cao. Nếu biết trước

với mỗi tổng s (tổng của tập con lấy từ các học sinh bên phải *low*) thì chỉ số lớn nhất của tập con đó là bao nhiêu, ta có thể ngay lập tức chọn được *high* tốt nhất cho *low* mà không cần vòng lặp *high*.

Cụ thể, ta xây dựng mảng **suffix best**: $best[i][s]$ là **chỉ số lớn nhất** của một tập con khác rỗng lấy từ các học sinh $i, i+1, \dots, N$ có tổng kỹ năng đúng bằng s ; nếu không tồn tại thì $best[i][s] = 0$. Nhờ đã sắp xếp theo P , chỉ số lớn nhất trong tập con cũng chính là đội trưởng của đội tạo từ tập đó. Như vậy, nếu ta cần một tập con phía sau *low* có tổng s để đưa vào đội của *high*, thì *high* ứng viên tốt nhất chính là $best[low+1][s]$.

Phần còn lại vẫn sử dụng **prefix difference DP** $possible[d]$ ($0 \leq d \leq T$) cho các học sinh trước *low*, với định nghĩa: $possible[d]$ đúng nếu có thể chia một số học sinh trước *low* thành hai đội mà trị tuyệt đối hiệu tổng là d . Khi duyệt *low* từ 1 đến N , ta duy trì mảng $possible$ này bằng cách thêm dần S_{low} (trước khi xét *low* thì $possible$ là trạng thái cho các học sinh $1..low-1$).

Điều kiện để $(low, high)$ khả thi: Tồn tại cách chọn tập trước *low* với hiệu d , và chọn tập sau *low* có tổng s để cân bằng:

$$sum_1 = X + S_{low}, \quad sum_2 = Y + s$$

với $|X - Y| = d$, $sum_1 = sum_2 \Rightarrow X - Y = s - S_{low}$. Do đó $|s - S_{low}| = d$, hay $s = S_{low} \pm d$ (giá trị s không âm). Khi đó $high = best[low+1][s]$. Nếu $high > 0$, ta cập nhật đáp án với $P_{high} - P_{low}$.

¶ Các bước của thuật toán.

- Sắp xếp học sinh theo P tăng dần, lưu chỉ số mới từ 1 đến N .
- Tính mảng $best$ bằng quy hoạch động từ phải sang trái (suffix knapsack):
 - Khởi tạo $best[i][s] = 0$ với mọi i, s .
 - Với $i = N \dots 1$:
 - Đầu tiên sao chép toàn bộ $best[i+1]$ sang $best[i]$ (bỏ qua i).
 - Chọn riêng mình i : $best[i][S_i] = \max(best[i][S_i], i)$.
 - Với mỗi $s \geq S_i$, nếu $best[i+1][s - S_i] > 0$, ta có thể gộp i vào tập đó, khi đó chỉ số lớn nhất của tập mới là $\max(i, best[i+1][s - S_i]) = best[i+1][s - S_i]$. Vậy $best[i][s] = \max(best[i][s], best[i+1][s - S_i])$.
- Duyệt *low* từ 1 đến N :
 - Với mỗi d mà $possible[d]$ đúng:
 - $req_1 = S_{low} + d$; nếu $req_1 \leq T$ và $best[low+1][req_1] > 0$, cập nhật $ans = \max(ans, P_{best[low+1][req_1]} - P_{low})$.
 - $req_2 = S_{low} - d$; nếu $req_2 \geq 0$ và $best[low+1][req_2] > 0$, cập nhật ans tương tự.
 - Cập nhật** $possible$ để chuẩn bị cho $low+1$: với mỗi d có $possible[d]$, ta thêm các trạng thái mới $d, d + S_{low}, |d - S_{low}|$ vào mảng tạm, rồi gán lại cho $possible$.
- In ans .

¶ **Chứng minh tính đúng đắn.** Với low cố định, phần prefix $possible[d]$ cho biết có thể tạo được hiệu d từ tập trước low . Phương trình cân bằng suy ra $s = S_{low} \pm d$. Nếu tồn tại tập hậu tố (từ $low + 1$ trở đi) có tổng s , thì $high = best[low + 1][s]$ chính là đội trưởng của tập đó. Vì $best$ lấy chỉ số lớn nhất, đây là ứng viên $high$ tối ưu cho low với hiệu d đó. Việc duyệt toàn bộ d đảm bảo xét hết mọi cách chia phần prefix, do đó toàn bộ các cặp $(low, high)$ hợp lệ đều được xem xét và ta tìm được độ hấp dẫn lớn nhất.

¶ **Ví dụ minh họa.** Sử dụng bộ dữ liệu mẫu: $S = [4, 7, 3, 8, 5, 6, 9, 2]$, $P = [15, 3, 20, 11, 8, 30, 6, 25]$. Sắp xếp theo P tăng dần:

- 1 : $P = 3, S = 7$
- 2 : $P = 6, S = 9$
- 3 : $P = 8, S = 5$
- 4 : $P = 11, S = 8$
- 5 : $P = 15, S = 4$
- 6 : $P = 20, S = 3$
- 7 : $P = 25, S = 2$
- 8 : $P = 30, S = 6$

$T = 44$.

Tính $best$ từ phải sang trái (truy vết một vài giá trị): $best[3][9] = 8$ (tập $\{S_6 = 3, S_8 = 6\}$ cho tổng 9, max index 8); $best[3][16] = 8$ (có thể $S_3 + S_4 + S_6 = 5 + 8 + 3 = 16$, max index 6, nhưng cũng có thể $S_7 + S_8 + S_4 = 2 + 6 + 8 = 16$, max index 8); $best[3][2] = 7$ (tập $\{S_7 = 2\}$).

Duyệt low từ 1 đến N :

- Ban đầu $possible[0] = \text{true}$.
- $low = 1$ ($S = 7, P = 3$): $possible = \{0\}$. $d = 0 \rightarrow req_1 = 7 + 0 = 7, req_2 = 7 - 0 = 7$. $best[2][7] = ?$ Từ $i = 2$ là $S = 9$ trở đi, có tổng 7 không? Các phần tử sau: 9, 5, 8, 4, 3, 2, 6 không có tổng 7 (vì 7 không xuất hiện). $best[2][7] = 0$. Cập nhật $possible$: thêm $S_1 = 7$: $\{0, 7, 7\} \rightarrow \{0, 7\}$.
- $low = 2$ ($S = 9, P = 6$): $possible = \{0, 7\}$. $d = 0$: $req_1 = 9, req_2 = 9$. $best[3][9] = 8 \neq 0 \rightarrow ans = \max(ans, 30 - 6 = 24)$. $d = 7$: $req_1 = 16, req_2 = 2$. $best[3][16] = 8 \rightarrow ans$ vẫn 24 ($30 - 6$). $best[3][2] = 7 \rightarrow 25 - 6 = 19 < 24$. Cập nhật $possible$: thêm $S_2 = 9$ vào $\{0, 7\}$ được $\{0, 9, 7, 16, |7 - 9| = 2\}$.
- Tiếp tục tăng low , ta sẽ còn tìm được các cặp khác nhưng không vượt quá 24. Kết quả cuối cùng $ans = 24$.

¶ **Lưu ý về kiểu dữ liệu.** P_i có thể lên đến 10^9 , hiệu $P_{high} - P_{low}$ nằm trong phạm vi $[-10^9, 10^9]$ nhưng đáp án luôn không âm và vừa với kiểu `int` 32 bit.

Mảng $best$ lưu chỉ số học sinh, vì $N \leq 1000$ nên dùng `short` (16 bit) để tiết kiệm bộ nhớ. Các mảng DP như $possible$ dùng `bool` cho hiệu quả.

Mảng $best$ kích thước $(N + 1) \times (T + 1)$ với `short` chiếm khoảng $1001 \times 100001 \times 2 \approx 200$ MB.

¶ Độ phức tạp.

- Thời gian: Tính *best* mất $O(N \cdot T)$ (hai vòng lặp lồng, mỗi ô được cập nhật $O(1)$). Vòng duyệt *low* cũng duyệt qua tất cả d từ 0 đến T , mỗi bước cập nhật *possible* trong $O(T)$, tổng cộng $O(N \cdot T)$. Vậy thời gian toàn bộ là $O(N \cdot T) \approx 1000 \cdot 100000 = 10^8$.
- Bộ nhớ: Mảng *best* $(N + 1) \times (T + 1)$ phần tử **short** tốn ~ 200 MB. Ngoài ra, các bạn có thể giảm bộ nhớ bằng cách chỉ giữ hai hàng liên tiếp (vì khi tính *best* từ phải sang trái, mỗi i chỉ cần hàng $i + 1$), hoặc tính *best* ngay trong lúc duyệt *low* giảm dần. Các bạn hãy thử cài thêm cách này nhé!

¶ Mã nguồn minh họa (C++).

```
1 #include <bits/stdc++.h> //NeOWami
2 using namespace std;
3
4 const int N = 1005;
5 const int T = 100005;
6
7 int n;
8 struct Student {
9     int S, P, id;
10     bool operator<(const Student& other) const {
11         return P < other.P;
12     }
13 };
14
15 Student a[N];
16 short best[N][T];
17 bool possible[T];
18
19 signed main() {
20     cin.tie(NULL)->sync_with_stdio(false); cout.tie(NULL);
21     if(ifstream("TEAMS.INP")) {
22         freopen("TEAMS.INP", "r", stdin);
23         freopen("TEAMS.OUT", "w", stdout);
24     }
25     cin >> n;
26
27     int max_T = 0;
28     for (int i = 1; i <= n; ++i) {
29         cin >> a[i].S;
30         max_T += a[i].S;
31     }
32     for (int i = 1; i <= n; ++i) {
33         cin >> a[i].P;
34         a[i].id = i;
35     }
36
37     sort(a + 1, a + 1 + n);
38     memset(best, 0, sizeof(best));
39
40     for (int i = n; i >= 1; --i) {
41         int S_i = a[i].S;
```

```
42     for (int s = 0; s <= max_T; ++s) best[i][s] = best[i + 1][s];
43
44     best[i][S_i] = max((short)best[i][S_i], (short)i);
45
46     for (int s = S_i; s <= max_T; ++s) {
47         if (best[i + 1][s - S_i] > 0) {
48             best[i][s] = max(best[i][s], best[i + 1][s - S_i]);
49         }
50     }
51 }
52
53
54 int ans = -1;
55 memset(possible, 0, sizeof(possible));
56 possible[0] = true;
57
58 for (int low = 1; low <= n; ++low) {
59     int S_low = a[low].S;
60     int P_low = a[low].P;
61
62     for (int d = 0; d <= max_T; ++d) {
63         if (possible[d]) {
64             int req1 = S_low + d;
65             if (req1 <= max_T && best[low + 1][req1] > 0) {
66                 ans = max(ans, a[best[low + 1][req1]].P - P_low);
67             }
68
69             int req2 = S_low - d;
70             if (req2 >= 0 && req2 <= max_T && best[low + 1][req2] >
71                 0) {
72                 ans = max(ans, a[best[low + 1][req2]].P - P_low);
73             }
74         }
75     }
76
77     bool next_possible[T] = {0};
78     for (int d = 0; d <= max_T; ++d) {
79         if (possible[d]) {
80             next_possible[d] = true;
81             next_possible[d + S_low] = true;
82             next_possible[abs(d - S_low)] = true;
83         }
84     }
85     for (int d = 0; d <= max_T; ++d) {
86         possible[d] = next_possible[d];
87     }
88
89     cout << ans << "\n";
90     return 0;
91 }
```

§3 Bài 3: Tuần Tra

§3.1 Đề bài

Tỉnh ZZZ có N thành phố được đánh số từ 0 đến $N - 1$, $N - 1$ con đường hai chiều tạo thành một cây. Khoảng cách $d(u, v)$ là số cạnh trên đường đi đơn giữa u và v . Hai cán bộ Nam và Huy xuất phát tại X và Y . Mỗi ngày, **đúng một** trong hai người di chuyển sang đỉnh kề. Chuyến tuần tra kết thúc khi cả hai đã đến mọi thành phố ít nhất một lần và quay về đúng vị trí xuất phát. Độ an toàn của chuyến đi là khoảng cách nhỏ nhất giữa hai người trong suốt hành trình (tính cả thời điểm ban đầu và sau mỗi lần di chuyển). Với mỗi truy vấn (X, Y) , hãy tìm **độ an toàn lớn nhất** có thể đạt được.

Dữ liệu vào

- Dòng đầu: Ghi 3 số S, N, Q ($1 \leq S \leq 5$, $1 \leq N \leq 2 \cdot 10^5$, $1 \leq Q \leq 10^5$);
- $N - 1$ dòng tiếp: Ghi A_i, B_i mô tả cạnh;
- Q dòng cuối: Ghi X, Y mô tả các truy vấn.

Kết quả

- Gồm Q dòng, mỗi dòng là đáp án của truy vấn tương ứng.

§3.2 Lời giải

§3.2.1 Nhận xét quan trọng

Gọi $d(u, v)$ là khoảng cách trên cây. Với mỗi đỉnh w , loại bỏ w sẽ chia cây thành các nhánh rời nhau. Ký hiệu $S_3(w)$ là độ dài lớn thứ ba trong số các nhánh đó (tính bằng số cạnh từ w đến điểm xa nhất trong nhánh). Nếu w có ít hơn ba nhánh, $S_3(w) = 0$. Đặt

$$L = \max_w S_3(w).$$

¶ **Tính chất 1 (chặn trên của độ an toàn).** Độ an toàn của mọi phương án không thể vượt quá L .

Chứng minh: Giả sử tồn tại phương án với độ an toàn $K > L$. Xét thời điểm cả hai người đã đi qua tất cả các đỉnh. Khi đó với mọi đỉnh w , cả hai đều đã ghé thăm mọi nhánh của w . Do mỗi ngày chỉ một người di chuyển, sẽ có lúc một người đi vào nhánh thứ ba trong khi người kia đang ở đâu đó. Nếu $K > L \geq S_3(w)$, thì khi một người đi sâu vào nhánh dài thứ ba (có độ dài $\leq L$), khoảng cách đến người còn lại (có thể đang ở nhánh khác hoặc tại w) sẽ có lúc nhỏ hơn hoặc bằng L , mâu thuẫn với việc khoảng cách luôn $\geq K > L$.

¶ **Tính chất 2 (đạt được L).** Tồn tại cách tổ chức để đạt độ an toàn đúng bằng L một khi hai người đã "thiết lập" được thể đứng phù hợp trong cây. Cụ thể, nếu chọn một đỉnh w có $S_3(w) = L$, ta có thể cho hai người luân phiên đứng yên tại các điểm sâu trong hai nhánh, trong khi người kia đi thăm nhánh thứ ba và quay lại, đảm bảo khoảng cách không bao giờ dưới L .

¶ **Ý tưởng chung.** Hành trình tối ưu sẽ gồm hai pha:

1. **Pha tiền tố (prefix):** Từ vị trí xuất phát (X, Y) , hai người di chuyển sao cho nhanh chóng đạt tới một cấu hình mà từ đó có thể duy trì khoảng cách L trong các bước sau. Trong pha này, khoảng cách nhỏ nhất đạt được gọi là `bestPrefix`.
2. **Pha duy trì:** Sau khi đã vào được "vùng an toàn", hai người sẽ thực hiện các bước di chuyển luân phiên qua các nhánh dài, bảo đảm khoảng cách không bao giờ nhỏ hơn L .

Đáp án cuối cùng là $\min(\text{bestPrefix}, L)$.

Bài toán quy về hai công việc: (i) Tính L ; (ii) Với mỗi cặp (X, Y) , tính `bestPrefix` – khoảng cách nhỏ nhất lớn nhất có thể đạt được trong quá trình "tiến vào" vùng ba nhánh. Việc tính `bestPrefix` được thực hiện bằng một quá trình mô phỏng tham lam trên cây, sẽ được mô tả ở các subtask sau.

§3.2.2 Subtask 1 (20% số điểm, $N, Q \leq 200$)

¶ **Tiếp cận.** Với mỗi truy vấn (X, Y) , ta tìm đáp án bằng cách thử lần lượt các giá trị K từ 0 đến $d(X, Y)$ (vì độ an toàn không thể lớn hơn khoảng cách ban đầu). Do tính đơn điệu: nếu đạt được độ an toàn K thì cũng đạt được mọi $K' \leq K$, ta có thể dùng tìm kiếm nhị phân.

Với một K , ta dựng đồ thị trạng thái: mỗi trạng thái là một cặp đỉnh (u, v) thể hiện vị trí hiện tại của Nam và Huy. Chỉ giữ lại các trạng thái có $d(u, v) \geq K$. Có cạnh nối giữa (u, v) và (u', v') nếu một trong hai người di chuyển sang đỉnh kề (tức u' kề u và $v' = v$, hoặc v' kề v và $u' = u$) và trạng thái mới vẫn thỏa mãn $d(u', v') \geq K$. Xuất phát từ (X, Y) , ta dùng BFS để xác định tất cả các trạng thái có thể đến được. Gọi U là tập đỉnh từng xuất hiện ở vị trí thứ nhất, V là tập đỉnh từng xuất hiện ở vị trí thứ hai. Nếu $U = V = \{0, 1, \dots, N-1\}$ thì tồn tại hành trình phủ hết các thành phố mà luôn giữ khoảng cách $\geq K$, đồng thời do đồ thị vô hướng và liên thông, ta có thể quay về (X, Y) . Khi đó K khả thi.

¶ **Chứng minh tính đúng đắn.** Mỗi trạng thái (u, v) khả thi tương ứng với việc sau một số ngày hợp lệ ta có thể đạt tới cấu hình đó. BFS cho ta tất cả cấu hình với khoảng cách $\geq K$ có thể đạt được. Điều kiện tập đỉnh xuất hiện đầy đủ đảm bảo mọi thành phố đều được ghé thăm.

¶ **Độ phức tạp.** Tiền xử lý tính $d(u, v)$ cho mọi cặp bằng N lần BFS/DFS: $O(N^2)$. Với mỗi truy vấn, chặt nhị phân $O(\log N)$ bước, mỗi bước BFS trên đồ thị trạng thái có tối đa N^2 đỉnh và $O(N^2 \cdot \Delta)$ cạnh (Δ là bậc tối đa), tốn $O(N^2)$. Toàn bộ: $O(N^2 + QN^2 \log N)$. Với $N, Q \leq 200$ là hoàn toàn đủ.

§3.2.3 Subtask 2 (20% số điểm, $N \leq 1000$)

¶ **Tiếp cận.** Ở Subtask 1, với mỗi truy vấn ta phải chặt nhị phân và BFS lại từ đầu, chi phí lên tới $O(Q \cdot N^2 \log N)$. Tuy nhiên, với $N \leq 1000$, số lượng trạng thái (u, v) chỉ là $N^2 = 10^6$. Thay vì giải từng truy vấn, ta có thể tiền xử lý đáp án cho mọi cặp đỉnh bằng bài toán "Đường đi có trọng số nhỏ nhất lớn nhất" trên đồ thị trạng thái.

¶ Thuật toán.

- Xây dựng đồ thị trạng thái G' : Mỗi đỉnh trong G' là một cặp vị trí (u, v) . Trọng số của đỉnh này chính là $d(u, v)$ (khoảng cách trên cây ban đầu).
- Có một cạnh nối giữa (u, v) và (u', v') nếu từ trạng thái này di chuyển hợp lệ sang trạng thái kia (một người đi 1 bước, người kia đứng yên).
- Ta cần tìm đường đi từ mọi (X, Y) đến nhóm các trạng thái "vùng an toàn" (các trạng thái (u, v) có thể đi vào 3 nhánh dài nhất) sao cho đỉnh có trọng số nhỏ nhất trên đường đi là lớn nhất.
- Bài toán này có thể giải bằng thuật toán Dijkstra cải tiến (thay vì tìm đường đi ngắn nhất, ta cập nhật đường đi có nút thắt cổ chai lớn nhất) xuất phát từ các trạng thái đích.
- Lưu kết quả vào mảng hai chiều $\text{ans}[X][Y]$. Khi trả lời truy vấn, ta chỉ việc in ra $\text{ans}[X][Y]$.

¶ **Độ phức tạp.** Số đỉnh của G' là $O(N^2)$, số cạnh là $O(N^2 \cdot \Delta) = O(N^2)$. Chạy thuật toán Dijkstra mất $O(N^2 \log(N^2))$. Với $N = 1000$, $N^2 \log(N^2) \approx 2 \times 10^7$ phép toán, hoàn toàn chạy tốt trong giới hạn thời gian. Truy vấn trả lời trong $O(1)$.

§3.2.4 Subtask 3 (20% số điểm, mọi truy vấn đều có khoảng cách xuất phát luôn đủ xa để đạt độ an toàn lớn nhất của cây)

¶ **Tiếp cận.** Khi biết trước mọi cặp (X, Y) đều cho đáp án bằng L (độ an toàn tối đa toàn cục), ta chỉ cần tính L và in ra cho tất cả truy vấn. Để tính $L = \max_w S_3(w)$, ta thực hiện rerooting DP (quy hoạch động đổi gốc) trên cây:

- Chọn gốc 0, tính độ sâu và mảng $\text{down}[v]$: độ dài đường đi dài nhất từ v xuống một nút trong cây con của nó.
- Tính $\text{outside}[v]$: độ dài đường đi dài nhất từ v đi lên phía cha và các nhánh khác. Khi chuyển từ v sang con u , $\text{outside}[u] = \max(\text{outside}[v], \text{sibling_max_down}) + 1$, với sibling_max_down là down lớn nhất trong các con khác của v .
- Với mỗi đỉnh w , ba nhánh dài nhất xuất phát từ w bao gồm: các giá trị $\text{down}[\text{child}] + 1$ với mỗi con, và giá trị $\text{outside}[w]$ (nếu w không phải gốc). Lấy độ dài lớn thứ ba trong số đó, gán vào $S_3(w)$. Cập nhật $L = \max(L, S_3(w))$.

¶ **Độ phức tạp.** $O(N)$ cho hai pha DFS. Mỗi truy vấn in L trong $O(1)$. Tổng $O(N + Q)$.

§3.2.5 Subtask 4 (20% số điểm, $Q \leq 200$)

¶ **Tiếp cận.** Với Q đủ nhỏ, ta có thể dùng thuật toán mô phỏng pha tiền tố với độ phức tạp $O(\sqrt{N} \log N)$ cho mỗi truy vấn. Cần tiền xử lý:

- Chọn gốc là một đỉnh opt_root bất kỳ đạt $S_3(\text{opt_root}) = L$ (đã tính được L và opt_root như subtask 3).
- Xây dựng mảng cha p_2 , độ sâu d , bảng tổ tiên $\text{up}[u][k]$ cho truy vấn LCA và nhảy xa.
- Với mỗi đỉnh u , tính $M[u]$ là đỉnh sâu nhất trong cây con gốc u (nếu nhiều đỉnh chọn đại diện), và $\text{maxD}[u] = d[M[u]]$.

¶ Mô phỏng pha tiền tố.

- Đặt $\xi = X$, $\eta = Y$, $z = \text{LCA}(X, Y)$. Khởi tạo $cur = +\infty$ (khoảng cách nhỏ nhất tích lũy), $\text{bestPrefix} = 0$.
- Trong mỗi bước lặp, ta xét xem có thể cải thiện bestPrefix bằng cách cho người đang ở độ sâu nhỏ hơn đi thẳng về z hay không: nếu $d[\xi] \leq d[\eta]$, $\text{bestPrefix} = \max(\text{bestPrefix}, \min(cur, d[\eta] - d[z]))$; công thức đối xứng cho trường hợp ngược lại.
- Xác định "bước chuyển nhánh": Với mỗi người, tìm $low[v]$ là tổ tiên gần nhất của v (có thể chính v) mà trong cây con của nó tồn tại một đỉnh có độ sâu lớn hơn $d[v]$. Nếu không có, $low[v] = -1$. Điều kiện để một bước chuyển hợp lệ là $low[v] \neq -1$ và $d[low[v]] > d[z]$ (còn nằm dưới z để tránh lợi ngược lên trên gây hẹp khoảng cách).
- Nếu cả hai đều có thể chuyển, ta chọn người có "khoảng cách đến nút rẽ nhánh" $d[v] - d[low[v]]$ nhỏ hơn (người đó có thể nhanh chóng đi vào nhánh dài). Người được chọn, giả sử ξ , sẽ di chuyển từ ξ tới $M[low[\xi]]$ (đi sâu nhất có thể trong nhánh đó). Khoảng cách nhỏ nhất trên đường đi này được cập nhật: $cur = \min(cur, d[\eta] + d[low[\xi]] - 2d[z])$. Cập nhật vị trí mới $\xi = M[low[\xi]]$.
- Lặp cho đến khi không còn ai có bước chuyển hợp lệ.
- Đáp án truy vấn: $\min(\text{bestPrefix}, L)$.

(Nếu $z = X$ hoặc $z = Y$, đáp án trực tiếp là $\min(d(X, Y), L)$.)

¶ **Tìm $low[v]$ trong mỗi bước.** Ta có thể dùng binary lifting: bắt đầu từ v , nhảy lên tổ tiên xa nhất có $maxD \leq d[v]$, $low[v]$ chính là cha của đỉnh đạt được (hoặc -1 nếu đỉnh đó là gốc và không có cha). Mỗi lần tìm mất $O(\log N)$. Do số bước mô phỏng là $O(\sqrt{N})$ (chứng minh ở subtask 5), tổng thời gian một truy vấn là $O(\sqrt{N} \log N)$.

¶ **Độ phức tạp.** Tiền xử lý $O(N \log N)$, Q truy vấn $O(Q\sqrt{N} \log N)$. Với $Q \leq 200$, $N = 2 \cdot 10^5$, chi phí này dễ dàng đáp ứng.

§3.2.6 Subtask 5 (20% số điểm, không ràng buộc thêm) – Lời giải đầy đủ

¶ **Ý tưởng / Tiếp cận.** Tối ưu từ subtask 4 bằng cách tiền xử lý toàn bộ $low[v]$ cho mọi đỉnh v trong $O(N \log N)$. Khi đó mỗi bước mô phỏng chỉ tốn $O(1)$ để lấy $low[v]$ và $O(1)$ để cập nhật cur , đưa độ phức tạp mỗi truy vấn về $O(\sqrt{N})$, đủ sức xử lý $Q = 10^5$. Nhắc lại: $low[v]$ là tổ tiên gần v nhất (có thể là chính v) sao cho trong cây con của nó có ít nhất một đỉnh sâu hơn $d[v]$. Để tính nhanh, với mỗi đỉnh u :

- Nếu $maxD[u] > d[u]$, đặt $low[u] = u$.
- Ngược lại, tìm tổ tiên p xa nhất của u thỏa mãn $maxD[p] \leq d[u]$ (dùng bảng up). Khi đó $low[u] = up[p][0]$ (cha của p), hoặc -1 nếu p là gốc và không có cha.

Việc này thực hiện một lần duy nhất trong $O(N \log N)$.

¶ Các bước của thuật toán.

- Tính L và chọn gốc opt_root :** Dùng rerooting DP (như subtask 3) để tìm $L = \max_w S_3(w)$ và một đỉnh opt_root đạt $S_3(\text{opt_root}) = L$.
- Xây dựng cây với gốc opt_root :** Mảng cha p_2 , độ sâu d , bảng nhảy $\text{up}[u][k]$ cho LCA và truy vấn tổ tiên.
- Tính $M[u]$ và $\text{maxD}[u]$:** Duyệt ngược thứ tự duyệt cây (từ lá lên gốc) để tìm đỉnh sâu nhất trong cây con.
- Tiền xử lý $\text{low}[u]$:** Với mỗi u , nếu $\text{maxD}[u] > d[u]$ thì $\text{low}[u] = u$; ngược lại nhảy ngược bằng binary lifting để tìm $\text{low}[u]$ (như mô tả trên), lưu vào mảng low .
- Xử lý truy vấn (X, Y) :**
 - Nếu $X = Y$ hoặc $z = X$ hoặc $z = Y$ (hai người nằm trên cùng một nhánh thẳng từ gốc): đáp án là $\min(d(X, Y), L)$.
 - Ngược lại, mô phỏng pha tiền tố:
 - $\xi \leftarrow X, \eta \leftarrow Y, z \leftarrow \text{LCA}(X, Y)$.
 - $\text{cur} \leftarrow +\infty, \text{bestPrefix} \leftarrow 0$.
 - Lặp:
 - Cập nhật $\text{cand} = \min(\text{cur}, d[\text{người sâu hơn}] - d[z])$, $\text{bestPrefix} = \max(\text{bestPrefix}, \text{cand})$.
 - Xác định $a_\xi = \text{low}[\xi], a_\eta = \text{low}[\eta]$.
 - $\text{valid}_\xi = (a_\xi \neq -1 \wedge d[a_\xi] > d[z])$, valid_η tương tự.
 - Nếu không ai hợp lệ: thoát lặp.
 - Chọn người di chuyển: nếu cả hai hợp lệ, chọn người có $d[v] - d[a_v]$ nhỏ hơn. Nếu một người hợp lệ, chọn người đó.
 - Giả sử ξ được chọn: $\text{cur} = \min(\text{cur}, d[\eta] + d[a_\xi] - 2d[z])$, $\xi \leftarrow M[a_\xi]$.
 - (Trường hợp η tương tự).
 - Đáp án: $\min(\text{bestPrefix}, L)$.

¶ Chứng minh tính đúng đắn.

- Pha tiền tố mô phỏng chiến lược tối ưu để hai người lần lượt "dọn" các nhánh dài chưa được thăm, mỗi lần một người đi sâu vào nhánh của mình trong khi người kia đứng yên. Việc chọn người có $d[v] - d[a_v]$ nhỏ hơn đảm bảo người đó nhanh chóng tiếp cận nhánh dài, giúp giữ khoảng cách lớn nhất có thể.
- Khoảng cách nhỏ nhất trên đường đi của người di chuyển so với người đứng yên chính là khoảng cách từ người đứng yên đến nút rẽ a_v , là $d[\text{stand}] + d[a_v] - 2d[z]$ (do z là LCA của cả hai vị trí hiện tại, và a_v nằm dưới z). Công thức này đúng vì đường đi giữa hai người đi qua z và a_v .
- Số bước lặp là $O(\sqrt{N})$: mỗi bước chọn một nhánh có độ sâu tăng thêm ít nhất là $d[v] - d[a_v] + (\text{maxD}[a_v] - d[a_v])$, các nhánh này không giao nhau và có kích thước tối thiểu là 1, 2, 3, ... (do ta ưu tiên nhánh ngắn trước), nên tổng số đỉnh được bao phủ ít nhất là $1 + 2 + \dots + k \leq N \Rightarrow k = O(\sqrt{N})$.

- Sau khi pha tiền tố kết thúc (không còn nhánh nào dưới z khả thi), hai người đã ở trong thế có thể duy trì khoảng cách L bằng cách luân phiên thăm các nhánh của opt_root (hoặc các đỉnh tương đương). Do đó đáp án bị chặn trên bởi L , và bestPrefix là mức an toàn đạt được tối đa trong pha đầu.

¶ Lưu ý về kiểu dữ liệu.

- $N, Q \leq 2 \cdot 10^5$, các giá trị khoảng cách có thể lên tới $N - 1$, cần dùng `int` là đủ (không quá $2 \cdot 10^5$). Biến `cur` ban đầu nên để bằng 10^9 .
- Mảng `up` cần kích thước $N \times LG$ với $LG = \lceil \log_2 N \rceil + 1 \approx 19$.

¶ Độ phức tạp.

- Tiền xử lý: Tính L và cây gốc mới $O(N)$. Xây bảng `up`, M , maxD , low : $O(N \log N)$.
- Mỗi truy vấn: LCA trong $O(\log N)$, số bước mô phỏng $O(\sqrt{N})$, mỗi bước $O(1)$. Tổng truy vấn $O(Q(\log N + \sqrt{N}))$.
- Bộ nhớ: $O(N \log N)$ cho bảng `up`, các mảng khác $O(N)$.

¶ Mã nguồn minh họa (C++).

```

1 #include <bits/stdc++.h> //NeOWami
2 using namespace std;
3
4 const int N = 2e5 + 5;
5 const int LG = 20;
6
7 int head[N], to[N * 2], nxt[N * 2], edge_cnt;
8 int pa[N], max_depth[N], out_depth[N], ord1[N];
9 int p2[N], d[N], ord2[N], maxD[N], M[N];
10 int up[N][LG], low[N];
11 int L_val, opt_root;
12
13 void add_edge(int u, int v) {
14     to[++edge_cnt] = v;
15     nxt[edge_cnt] = head[u];
16     head[u] = edge_cnt;
17 }
18
19 int get_lca(int u, int v) {
20     if (d[u] < d[v]) swap(u, v);
21     for (int i = LG - 1; i >= 0; --i)
22         if (d[u] - (1 << i) >= d[v])
23             u = up[u][i];
24     if (u == v) return u;
25     for (int i = LG - 1; i >= 0; --i)
26         if (up[u][i] != up[v][i]) {
27             u = up[u][i];
28             v = up[v][i];
29         }
30     return up[u][0];
31 }
```

```
32
33 void compute_L_and_root(int n) {
34     int fr = 0, bk = 0;
35     ord1[bk++] = 0; pa[0] = 0;
36     while (fr < bk) {
37         int u = ord1[fr++];
38         for (int e = head[u]; e; e = nxt[e]) {
39             int v = to[e];
40             if (v != pa[u]) {
41                 pa[v] = u;
42                 ord1[bk++] = v;
43             }
44         }
45     }
46     pa[0] = -1;
47
48     for (int i = n - 1; i >= 0; --i) {
49         int u = ord1[i];
50         max_depth[u] = 0;
51         for (int e = head[u]; e; e = nxt[e]) {
52             int v = to[e];
53             if (v != pa[u])
54                 max_depth[u] = max(max_depth[u], max_depth[v] + 1);
55         }
56     }
57
58     for (int i = 0; i < n; ++i) {
59         int u = ord1[i];
60         int m1 = 0, m2 = 0;
61         for (int e = head[u]; e; e = nxt[e]) {
62             int v = to[e];
63             if (v != pa[u]) {
64                 int val = max_depth[v] + 1;
65                 if (val > m1) m2 = m1, m1 = val;
66                 else if (val > m2) m2 = val;
67             }
68         }
69         for (int e = head[u]; e; e = nxt[e]) {
70             int v = to[e];
71             if (v != pa[u]) {
72                 int val = max_depth[v] + 1;
73                 int sibling_max = (val == m1) ? m2 : m1;
74                 out_depth[v] = max(out_depth[u], sibling_max) + 1;
75             }
76         }
77     }
78
79     L_val = 0; opt_root = 0;
80     int b[3];
81     for (int u = 0; u < n; ++u) {
82         b[0] = b[1] = b[2] = -1;
83         auto add_branch = [&](int val) {
84             if (val > b[0]) b[2]=b[1], b[1]=b[0], b[0]=val;
```

```
85         else if (val > b[1]) b[2]=b[1], b[1]=val;
86         else if (val > b[2]) b[2]=val;
87     };
88     if (pa[u] != -1) add_branch(out_depth[u]);
89     for (int e = head[u]; e; e = nxt[e]) {
90         int v = to[e];
91         if (v != pa[u]) add_branch(max_depth[v] + 1);
92     }
93     int s3 = (b[2] != -1) ? b[2] : 0;
94     if (s3 >= L_val) {
95         L_val = s3;
96         opt_root = u;
97     }
98 }
99 }
100
101 void build_with_root(int n) {
102     int fr = 0, bk = 0;
103     ord2[bk++] = opt_root;
104     p2[opt_root] = opt_root; d[opt_root] = 0;
105     while (fr < bk) {
106         int u = ord2[fr++];
107         for (int e = head[u]; e; e = nxt[e]) {
108             int v = to[e];
109             if (v != p2[u]) {
110                 p2[v] = u;
111                 d[v] = d[u] + 1;
112                 ord2[bk++] = v;
113             }
114         }
115     }
116     p2[opt_root] = -1;
117
118     for (int i = n - 1; i >= 0; --i) {
119         int u = ord2[i];
120         maxD[u] = d[u];
121         M[u] = u;
122         for (int e = head[u]; e; e = nxt[e]) {
123             int v = to[e];
124             if (v != p2[u]) {
125                 if (maxD[v] > maxD[u]) {
126                     maxD[u] = maxD[v];
127                     M[u] = M[v];
128                 }
129             }
130         }
131     }
132
133     for (int i = 0; i < n; ++i) {
134         up[i][0] = p2[i];
135         for (int j = 1; j < LG; ++j) up[i][j] = -1;
136     }
137     for (int j = 1; j < LG; ++j)
```

```
138     for (int i = 0; i < n; ++i)
139         if (up[i][j - 1] != -1)
140             up[i][j] = up[up[i][j - 1]][j - 1];
141
142     for (int u = 0; u < n; ++u) {
143         if (maxD[u] > d[u]) {
144             low[u] = u;
145         } else {
146             int curr = u;
147             for (int i = LG - 1; i >= 0; --i) {
148                 int nxt = up[curr][i];
149                 if (nxt != -1 && maxD[nxt] <= d[u])
150                     curr = nxt;
151             }
152             low[u] = up[curr][0];
153         }
154     }
155 }
156
157 int query(int X, int Y) {
158     int z = get_lca(X, Y);
159     if (z == X || z == Y)
160         return min(d[X] + d[Y] - 2 * d[z], L_val);
161     int cur = 1e9, bestPrefix = 0;
162     int xi = X, eta = Y;
163     while (true) {
164         int cand = (d[xi] <= d[eta]) ? min(cur, d[eta] - d[z])
165                                     : min(cur, d[xi] - d[z]);
166         bestPrefix = max(bestPrefix, cand);
167         int a_xi = low[xi], a_eta = low[eta];
168         bool valid_xi = (a_xi != -1 && d[a_xi] > d[z]);
169         bool valid_eta = (a_eta != -1 && d[a_eta] > d[z]);
170         if (!valid_xi && !valid_eta) break;
171         bool move_xi;
172         if (valid_xi && valid_eta)
173             move_xi = (d[xi] - d[a_xi] < d[eta] - d[a_eta]);
174         else
175             move_xi = valid_xi;
176         if (move_xi) {
177             cur = min(cur, d[eta] + d[a_xi] - 2 * d[z]);
178             xi = M[a_xi];
179         } else {
180             cur = min(cur, d[xi] + d[a_eta] - 2 * d[z]);
181             eta = M[a_eta];
182         }
183     }
184     return min(bestPrefix, L_val);
185 }
186
187 signed main() {
188     cin.tie(NULL)->sync_with_stdio(false); cout.tie(NULL);
189     if (ifstream("PATROL.INP")) {
190         freopen("PATROL.INP", "r", stdin);
```

```
191     freopen("PATROL.OUT", "w", stdout);
192 }
193
194 int S, n, q;
195 cin >> S >> n >> q;
196 for (int i = 1; i < n; ++i) {
197     int u, v; cin >> u >> v;
198     add_edge(u, v); add_edge(v, u);
199 }
200 if (n == 1) {
201     while (q--) { int x, y; cin >> x >> y; cout << "0\n"; }
202     return 0;
203 }
204 compute_L_and_root(n);
205 build_with_root(n);
206 while (q--) {
207     int X, Y; cin >> X >> Y;
208     cout << query(X, Y) << '\n';
209 }
210 return 0;
211 }
```